HYUNDAI AI Insight Campus · 온디바이스 AI

Day 2 GPIO 출력 · 입력 · 이벤트

Embedded I/O 제어 2/5 · 10/14(수) · Raspberry Pi 4 + libgpiod 2.x (C)

강의 자료 · 애니메이션 · 워크시트

오늘 만드는 것

Day 1 → Day 2

gpioset · gpioget · gpiomon 명령으로 하던 일을
libgpiod v2 C 함수로 직접 한다 .

미니 프로젝트 — react.c

무작위 지연 → LED → 버튼 반응 시간 (ms)
부정 출발 · 2 인 대전 · CSV 기록

1	09:15	**libgpiod v2 구조와 첫 출력**
2	10:00	**여러 라인 동시 출력**
3	11:00	**레지스터 직접 접근 비교**
4	13:00	**디지털 입력과 bias**
5	14:00	**엣지 이벤트와 poll()**
6	15:00	**여러 입력 · 타이머 한 루프로**
7	16:00	**미니 프로젝트 설계**
8	17:00	**미니 프로젝트 구현**

오늘 쓰는 핀

BCM (물리)	부품	교시
GPIO17 (11) · GPIO27 (13) · GPIO22 (15)	LED 빨강 · 노랑 · 초록	1 · 2 · 3 · 7 · 8
GPIO5 (29)	버튼 A	3 ~ 8
GPIO6 (31)	버튼 B → 기울기 센서 → PIR → 버튼 B	4 · 6 · 8 (바꿔 꽂음)
GPIO13 (33)	액티브 부저	7 · 8

공통 헤더 gpio_common.h

SIGINT 처리 · now_ns() · sleep_ms() · gpio_request_output() / gpio_request_input() — Day 3~5 에서도 그대로 쓴다 .

명령에서 C 함수로

1 교시

Day 1 명령	Day 2 C 함수
gpioset 17=1 (기다림)	gpiod_chip_request_lines() → gpiod_line_request_set_value()
gpioget -b pull-up 5	입력 · bias 설정 후 gpiod_line_request_get_value()
gpiomon -e falling 5	edge 설정 후 poll() + gpiod_line_request_read_edge_events()
Ctrl+C → 라인 해제	gpiod_line_request_release()

명령 도구도 안에서는 같은 libgpiod 함수를 부른다 .

인터넷 예제 주의 — v1 과 v2 는 C API 가 완전히 다르다

gpiod_line_request_output() 같은 v1 함수가 보이면 이 환경에서는 컴파일되지 않는다 . v2 예제에는 gpiod_request_config 가 있다 .

libgpiod v2 — 재료를 쌓아 한 번에 요청

1교시

① chip
gpiod_chip_open

② settings
방향 · 초기값 · bias · edge

③ line_config
offset ↔ settings

④ request_config
consumer 이름

⑤ request
커널에 요청 → 소유

⑥ release
반납

②~④ 는 내 메모리의 재료

커널은 아직 모른다 . 요청 직후 free 해도 된다 .
같은 settings 를 여러 라인에 재사용 .

⑤ 에서 처음 커널로

성공하면 이 프로세스가 소유자 — gpioinfo 에 consumer, 다른 요청은 busy.
요청만 있으면 칩은 닫아도 된다 .

애니메이션 D2-1 • 객체 흐름과 반납 ▶

Ctrl+C 를 잡아 정리하고 나가기

1교시

```
static volatile sig_atomic_t g_stop = 0;
static void on_sigint(int s) { g_stop = 1; }

sa.sa_handler = on_sigint;   /* SA_RESTART 없음 */
sigaction(SIGINT, &sa, NULL);

while (!g_stop) { ... }
set_value(req, 17, INACTIVE);
gpiod_line_request_release(req);
```

종료 방법	라인	LED 끄기
Ctrl+C (핸들러 있음)	반납	실행됨
Ctrl+C (핸들러 없음)	커널이 해제	실행 안 됨
kill -9	커널이 해제	실행 안 됨

SA_RESTART 를 주지 않아야 poll · nanosleep 이 EINTR 로 깨어나 루프가 g_stop 을 확인한다 .

라인 세 개를 한 요청으로 — 신호등 상태표

2 교시

```
LINES[3] = { 17, 27, 22 };   /* R Y G */
req = gpio_request("traffic", LINES, 3, s);

v[3] = { R, Y, G };
gpiod_line_request_set_values(req, v);
```

#	상태	R	Y	G	ms
0	초록	0	0	1	3000
1	노랑	0	1	0	1000
2	빨강	1	0	0	3000
3	빨강 + 노랑	1	1	0	1000

set_values() 의 배열 순서 = 요청할 때 넘긴 offsets 순서

세 LED 를 시스템 콜 한 번으로 동시에 바꾼다 . 순서를 바꾸려면 코드가 아니라 표만 고친다 .

애니메이션 D2-2 • 신호등과 active-low ▶

active-low 와 drive

2 교시

설정	ACTIVE (1) →	INACTIVE (0) →	쓰는 곳
active-high (기본)	3.3 V	0 V	GPIO → LED → GND (오늘 배선)
active-low	0 V	3.3 V	3V3 → LED → GPIO, Low 동작 모듈

drive	1 을 쓰면	0 을 쓰면	쓰는 곳
PUSH_PULL (기본)	3.3 V 로 밀어냄	0 V	LED · 부저
OPEN_DRAIN	손을 놓음 (외부 풀업)	0 V	한 선 공유 — I2C (Day 4)

active-low 면 코드는 "1 = 켜짐 " 그대로 , 전압 반전은 커널이 한다 .

BCM2711 GPIO 레지스터 — /dev/gpiomem

3 교시

오프셋	이름	한 칸	쓰는 법
0x00~0x14	GPFSEL0~5	핀당 3 비트 기능 (000 입력 · 001 출력)	읽기 - 수정 - 쓰기
0x1C	GPSET0	1 을 쓴 핀 = High	쓰기 한 번
0x28	GPCLR0	1 을 쓴 핀 = Low	쓰기 한 번
0x34	GPLEV0	핀 현재 레벨	읽기
0xE4~	PUP_PDN_CNTRL	핀당 2 비트 pull	읽기 - 수정 - 쓰기

```
/* GPIO17 = 출력 */
/* GPFSEL1, 비트 21~23 */
v = gpio[1];
v &= ~(7u << 21);
v |=  (1u << 21);
gpio[1] = v;

gpio[0x1C/4] = 1u << 17;
gpio[0x28/4] = 1u << 17;
```

번호 = pin / 10 · 시작 비트 = (pin % 10) × 3 · SET/CLR/LEV 비트 = pin

애니메이션 D2-3 • 레지스터 비트 계산기 ▶

mmap 은 수십 배 빠르다 — 그래도 평소엔 안 쓴다

3 교시

약 1 µs

libgpiod 토글 1 회
(ioctl 시스템 콜)

약 15 ns

mmap 토글 1 회
(메모리 쓰기)

mmap 의 문제	설명
소유권 없음	gpioinfo 에 안 보임 · busy 보호 없음
이식성 없음	Pi 5(RP1) 는 레지스터가 전혀 다름
인터럽트 없음	엣지를 보려면 계속 읽어야 함
정리 책임	죽으면 핀이 출력으로 남음

숫자는 예시 — 04_speed 로 직접 측정 . 정확한 파형은 HW PWM · SPI (Day 3 · 4).

입력에는 bias 가 필요하다

4 교시

bias	뜻	Arduino
PULL_UP	내부 저항 → 3.3V	INPUT_PULLUP
PULL_DOWN	내부 저항 → GND	INPUT_PULLDOWN (ESP32)
DISABLED	저항 끔 — 센서가 직접 0/1	INPUT
AS_IS	지금 상태 그대로 (기본값)	—

버튼 (GND 쪽)

PULL_UP — 평소 1, 누르면 0

PIR (센서 출력)

DISABLED — 센서가 직접 0/1

폴링 + 소프트웨어 디바운스

4 교시

```
v = gpiod_line_request_get_value(req, 5);
same = (v == last) ? same + 1 : 0;
last = v;
if (same >= 20 && v != stable) {   /* 20 ms 안정 */
    stable = v;
    if (v == 0) printf("눌림\n");
}
sleep_ms(1);
```

폴링 간격	1 초당 시스템 콜	놓치는 것
1 ms	약 1000	1 ms 미만 펄스
10 ms	약 100	10 ms 미만 펄스
sleep 없음	수십만	— (CPU 100%)

기울기 센서 : ./05_button_poll -n 6 — 바운싱이 버튼보다 훨씬 심하다

커널 디바운스

gpiod_line_settings_set_debounce_period_us()
카운터 없이 커널이 걸러 준다 → 5 교시부터 사용

애니메이션 D2-4 • 폴링 간격과 디바운스 ▶

엣지 이벤트 — 커널이 알려 줄 때까지 잠든다

5 교시

```
req = gpio_request_input(..., EDGE_BOTH, 10000);
pfd.fd = gpiod_line_request_get_fd(req);

poll(&pfd, 1, -1);                /*  잠듦  */
n = gpiod_line_request_read_edge_events(req, buf, 16);
ev = gpiod_edge_event_buffer_get_event(buf, i);
type = gpiod_edge_event_get_event_type(ev);
ts   = gpiod_edge_event_get_timestamp_ns(ev);
```

	폴링	이벤트
CPU	1~2 %	0 %
시각	읽은 순간	커널 ts (ns)
짧은 펄스	놓침	큐에 쌓임

타임스탬프 = CLOCK_MONOTONIC → now_ns() 와 바로 뺄 수 있다 .

애니메이션 D2-5 • busy loop vs poll ▶

ESP32 인터럽트 vs Linux 엣지 이벤트

5 교시

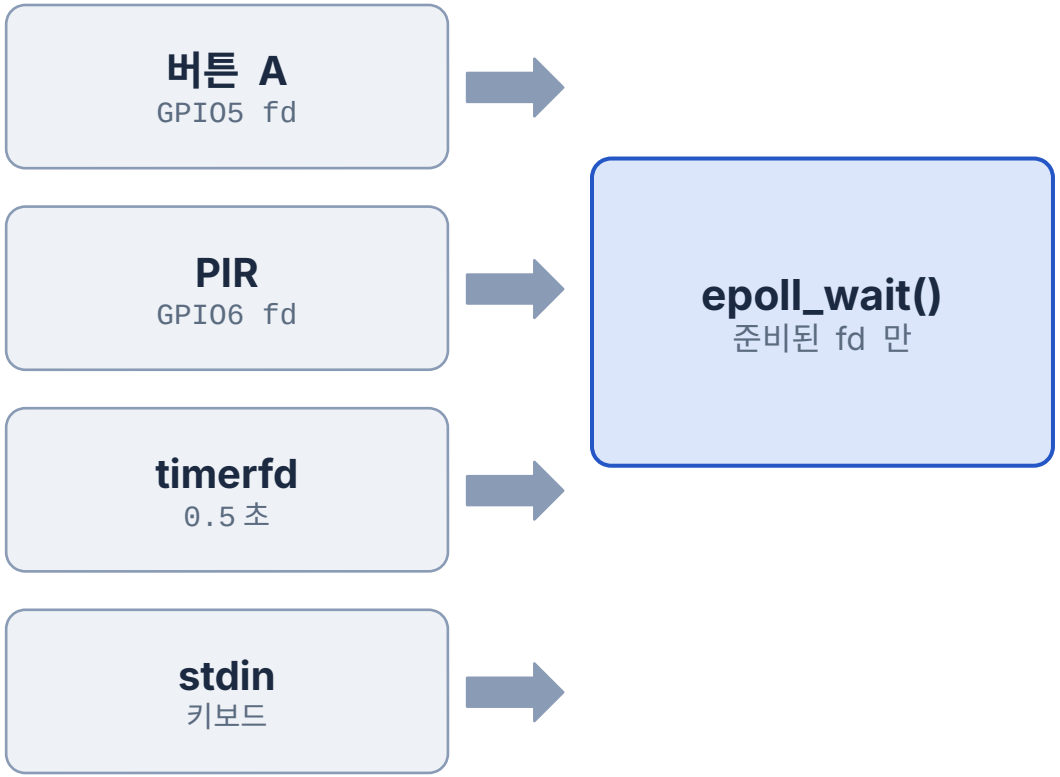
	ESP32 attachInterrupt	Linux 엣지 이벤트
내 코드가 도는 곳	ISR (인터럽트 문맥)	일반 프로세스
제약	짧게 · volatile · IRAM_ATTR · printf 금지	없음
지연	수 µs	수십 µs ~ ms
시각	ISR 에서 micros()	커널이 인터럽트 순간 찍음

지연은 늘었지만 " 언제 눌렸나 " 는 여전히 정확하다

→ 7 · 8 교시 반응속도 게임이 이 타임스탬프를 쓴다 . 짧게 / 길게 누름 = rising ts – falling ts.

epoll — 버튼 · PIR · 타이머 · 키보드를 한 루프로

6 교시




```
tfd = timerfd_create(CLOCK_MONOTONIC, 0);
timerfd_settime(tfd, 0, &its, NULL);
ep = epoll_create1(0);
epoll_ctl(ep, EPOLL_CTL_ADD, fd, &e);  /* ×4 */

n = epoll_wait(ep, evs, 4, -1);
switch (evs[i].data.u32) {
 case TAG_BTN:   read_edge_events …
 case TAG_TIMER: read(tfd, &exp, 8) …
}
```

준비된 fd 를 read 하지 않으면 epoll_wait 가 계속 즉시 반환 → busy loop

애니메이션 D2-6 • epoll 이벤트 루프 ▶

PIR 배선과 timerfd

6 교시

PIR 핀	연결	주의
VCC	3V3 (물리 1 또는 17)	5V 면 OUT 이 3.3V 를 넘을 수 있음
OUT	GPIO6 (버튼 B 자리)	bias DISABLED
GND	GND 레일	전원 후 30~60 초 안정화

timerfd 함수	하는 일
timerfd_create(CLOCK_MONOTONIC, 0)	타이머 fd 생성
timerfd_settime(fd, 0, &its, NULL)	it_value 첫 만료 · it_interval 주기
read(fd, &exp, 8)	만료 횟수 — 읽어야 다시 대기

반응속도 게임 — 상태 머신

7 교시

READY
" 준비 " · drain · 2~5 초 대기

GO
LED 켜고 t_on 기록 · 3 초 대기

RESULT
ms = (ts − t_on) / 1e6 · 부저 0.1 초

FOUL
지연 중 누름 · 부저 0.8 초

```
/* 반환 : 1 = 눌림 (*ts, *who), 0 = 시간 초과 , -1 = Ctrl+C */
static int wait_press(int timeout_ms, long long *ts, unsigned int *who);
```

애니메이션 D2-7 • 반응속도 게임 ▶

배선과 시간 재는 법

7 교시

BCM	부품
GPIO17	LED
GPIO5	버튼 A
GPIO6	버튼 B (2 인)
GPIO13	액티브 부저 (+)

방법	계산	오차
❌ 깨어난 뒤	now_ns() − t_on	스케줄러 지연이 더해짐
✅ 커널 타임스탬프	timestamp_ns(ev) − t_on	인터럽트 순간 — 부하와 무관

drain() 이 필요한 이유

이전 라운드에서 뗄 때의 이벤트 · 바운싱이 큐에 남아 있으면 다음 라운드 지연 중에 바로 읽혀 " 부정 출발 " 이 된다 .

react.c 핵심

8 교시

```
drain();
int delay = 2000 + rand() % 3000;
if (wait_press(delay, &ts, &who) == 1) {
    fouls++;  beep(800);  continue;     /* 부정 출발 */
}
set_line(LED, 1);
long long t_on = now_ns();
if (wait_press(3000, &ts, &who) == 1) {
    long ms = (ts - t_on) / 1000000;
    beep(100);
}
set_line(LED, 0);
```

결과	예
1 인	성공 4 · 평균 219 · 최고 198 ms
2 인 (-2)	A 2 : 3 B (line offset)
CSV	react_log.csv 에 "a" 로 추가

부하 실험

yes > /dev/null & ×4

→ 기록이 거의 변하지 않음

killall yes 로 정리

애니메이션 D2-8 • 타임스탬프 vs 깨어난 시각 ▶

오늘 정리

요청 → 사용 → 반납 · offsets 순서 = 값 순서 · mmap 은 빠르지만 위험 · 입력엔 bias · 이벤트 + 커널 타임스탬프 · epoll 하나로 여러 입력

다음 시간 (Day 3, 10/15) 소프트웨어 PWM jitter · 하드웨어 PWM(/sys/class/pwm) · 서보 · 부저 · DC 모터 · 스테퍼 · 초음파 거리 경보기

워크시트 풀기 ▶